CTF Writeup

Rooted or not? — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Rooted or not? **Author:** LordSudo **Difficulty:** Easy **Points:** 100 **Category:** Mobile / Android — Root Detection Bypass **Result:** Solved (1/1 flag captured)

Description

> A vault app exposes a debug panel — but the developers claim it's safe since it's gated behind a root check, and "real users don't root their phones."

Vulnerability

Client-side-only "protection". The flag is XOR-encoded in the APK and decrypted by `RootCheck.unlockDebugFlag()`, whose algorithm is fully visible in smali. No device, no root, no runtime — the flag is recoverable by pure static analysis. (CWE-693: Protection Mechanism Failure.)

Flag

`r00t{st4t1c_4n4lys1s_byp4ss3s_th3_ch3ck}`

Recon

The APK contained two classes: - `com.example.vaultapp.MainActivity` - `com.example.vaultapp.RootCheck`

`MainActivity.onCreate` calls `RootCheck.isDeviceRooted()`: - if rooted → logs "Root detected - access denied" - if not root → calls `RootCheck.unlockDebugFlag()` and logs the flag

The devs assumed an attacker couldn't reach the debug panel. Wrong: the debug logic runs entirely on the client, so we can just re-implement it offline.

The Decryption Logic (from smali)

`RootCheck.smali` contains:

- A 40-byte array `ENCODED_FLAG` (the ciphertext), set in `<clinit>` via `fill-array-data`.
- A constant `SEED = 0x5A`.
- `deriveKey(int length)` builds the keystream: key[i] = (0x5A + i * 7) & 0xFF
- `unlockDebugFlag()` XORs each ciphertext byte with the corresponding key byte: plaintext[i] = ENCODED_FLAG[i] ^ key[i]
- Returns the result as a `String`.

Extraction & Decode

Dumped the smali with apktool: apktool d -f vaultapp.apk -o out

Read the encoded array and the key derivation: cat out/smali/com/example/vaultapp/RootCheck.smali

Re-implemented the XOR in Python:

```
enc = [0x28,0x51,0x58,0x1b,0x0d,0x0e,-0x10,-0x41,-0x1a,-0x58,-0x3d,-0x08,
       -0x66,-0x25,-0x78,-0x51,-0x4d,-0x5e,-0x17,-0x54,-0x47,-0x71,-0x73,-0x75,
       0x36,0x7a,0x63,0x24,0x6d,0x7a,0x58,0x5b,0x09,0x1e,0x2b,0x27,0x65,0x3e,0x0f,0x16]
enc = [b & 0xFF for b in enc]
key = [(0x5a + i*7) & 0xFF for i in range(len(enc))]
print(''.join(chr(e ^ k) for e, k in zip(enc, key)))
```

Output: r00t{st4t1c_4n4lys1s_byp4ss3s_th3_ch3ck}

Issues Hit & How They Were Solved

1. `apktool d *.apk` **failed** — "Input file (*.apk) was not found". Cause: the archive extracted into a subfolder with a space ("Rooted or not/"), so the glob never matched at the current level. Fix: `cd "Rooted or not"` first, then reference the APK by name.
2. **No plaintext flag in `strings` or resource grep.** Cause: the flag is XOR-encoded, not stored as a literal. Fix: read the smali to recover the algorithm instead of searching for the literal.
3. **Root check looked like a barrier.** Cause: misdirection — the description framed the root check as the wall. Fix: recognize that the flag is produced by a pure function of constants inside the APK. Running the app was never necessary.

Key Takeaways

- Client-side "protections" (root checks, anti-debug, obfuscation) never protect a secret — they only delay a determined analyst.
- XOR with a deterministic key stream is not encryption; once the key formula is visible in decompiled code, the plaintext is one script away.
- Real fix: never ship the secret to the client. Fetch/derive it server-side, or don't expose the debug path in release builds at all.

Tools Used

apktool, grep, strings, python3